ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ В РАЗРАБОТКЕ ИГР 2
ЧАСТЬ 1: ОСНОВЫ 2
1. 1 Что такое паттерны и для чего они нужны 2
1.2 SOLID (Кратко — фундамент паттернов) 2
ЧАСТЬ 2: ПОРОЖДАЮЩИЕ ПАТТЕРНЫ (Creational) 2
2.1. Фабричный Метод (Factory Method) 2
2.2. Абстрактная Фабрика (Abstract Factory) 5
2.3. Строитель (Builder) 7
2.4. Одиночка (Singleton) 9
2.5. Прототип (Prototype) 11
ЧАСТЬ 3: СТРУКТУРНЫЕ ПАТТЕРНЫ (Structural) 12
3.6. Адаптер (Adapter) 13
3.7. Мост (Bridge) 15
3.8. Компоновщик (Composite) 18
3.9. Декоратор (Decorator) 21
10. Фасад (Facade) 25
11. Легковес (Flyweight) 27
12. Заместитель (Proxy) 30
ЧАСТЬ 4: ПОВЕДЕНЧЕСКИЕ ПАТТЕРНЫ (Behavioral) 32
4.13. Цепочка Обязанностей (Chain of Responsibility) 33
4.14. Команда (Command) 35
4.15. Итератор (Iterator) 38
4.16. Посредник (Mediator) 40
4.17. Снимок (Memento) 43
4.18. Наблюдатель (Observer) 46
4.19. Состояние (State) 49
4.20. Шаблонный Метод (Template Method) 53

21. Посетитель (Visitor) 55
ИТОГОВАЯ ТАБЛИЦА “ПРОБЛЕМА → ПАТТЕРН” ДЛЯ ИГР 59
КЛЮЧЕВЫЕ ПРИНЦИПЫ ОБУЧЕНИЯ (Повторение) 60

Ниже раскрываются паттерны проектирования с фокусом на **проблему → решение**, **игровую тематику** и параллели между **псевдокодом, C++ и C#**.

ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ В РАЗРАБОТКЕ ИГР

ЧАСТЬ 1: ОСНОВЫ

1. 1 Что такое паттерны и для чего они нужны

Проблема:
Вы пишете код игры. Через месяц возвращаетесь к нему и не понимаете, что там происходит. Добавление новой фичи ломает старые. Один класс знает про всех и всё.

Идея:
Паттерны — это проверенные решения типовых проблем архитектуры. Это не готовый код, а **шаблон мышления**.

Аналогия:
Архитектор не рисует каждый дом с нуля — он использует типовые решения: арки, колонны, несущие стены. Паттерны — такие же строительные блоки для кода.

1.2 SOLID (Кратко — фундамент паттернов)

- **S**ingle Responsibility: Класс делает только одно (Moves, Attacks, но не всё сразу)
- **O**pen/Closed: Класс открыт для расширения, закрыт для изменения (добавляем новое оружие, не трогая старый код)
- **L**iskov Substitution: Наследники дополняют, а не ломают родителя
- **I**nterface Segregation: Лучше много мелких интерфейсов, чем один большой
- **D**ependency Inversion: Зависеть от абстракций, а не от конкретных классов

ЧАСТЬ 2: ПОРОЖДАЮЩИЕ ПАТТЕРНЫ (Creational)

О том, **как создавать объекты**.

2.1. Фабричный Метод (Factory Method)

Проблема:
В игре нужно создавать врагов. Сейчас код забит if-ами по типу врага.

```
// Плохой код
Enemy CreateEnemy(string type)
{
   if (type == "Goblin") return new Goblin();
   if (type == "Orc") return new Orc();
   if (type == "Dragon") return new Dragon();
   return null;
}
```

Каждый раз при добавлении нового врага (Troll) придется лезть в этот метод и менять его.

Идея:
Вынести создание объектов в отдельный метод, который переопределяют наследники.

Псевдокод:

```
класс Уровень
   метод СоздатьВрага()   // фабричный метод (абстрактный)

класс ЛесноеУровень : Уровень
   метод СоздатьВрага()
      возвращать new Гоблин()

класс ВулканическийУровень : Уровень
   метод СоздатьВрага()
      возвращать new Дракон()
```

C++ (MonsterFactory):

```
// Абстрактный класс врага
class Monster {
public:
   virtual void Roar() = 0;
};

class Goblin : public Monster {
public:
   void Roar() override { cout << "Гоблин пищит!" << endl; }
};
```

```
class Dragon : public Monster {
public:
    void Roar() override { cout << "ДРАКОН РЕВЕТ!" << endl; }
};

// Фабричный метод
class Level {
public:
    // Это и есть фабричный метод
    virtual Monster* CreateMonster() = 0;

    void SpawnAndRoar() {
        Monster* m = CreateMonster(); // Вызов фабричного метода
        m->Roar();
        delete m; // упрощенно
    }
};

class ForestLevel : public Level {
public:
    Monster* CreateMonster() override {
        return new Goblin(); // создаем конкретный тип
    }
};

class VolcanoLevel : public Level {
public:
    Monster* CreateMonster() override {
        return new Dragon();
    }
};

// Использование
Level* level = new ForestLevel();
level->SpawnAndRoar(); // Выведет: "Гоблин пищит!"
```

C#:

```
// Аналогично C++, но с интерфейсами
interface IMonster
{
    void Roar();
}
```

```
class Goblin : IMonster
{
   public void Roar() => Console.WriteLine("Гоблин пищит!");
}

abstract class Level
{
   public abstract IMonster CreateMonster(); // Фабричный метод

   public void SpawnAndRoar()
   {
      IMonster monster = CreateMonster();
      monster.Roar();
   }
}

class ForestLevel : Level
{
   public override IMonster CreateMonster() => new Goblin();
}
```

2.2. Абстрактная Фабрика (Abstract Factory)

Проблема:
В игре есть расы: Люди и Орки. У каждой расы свой солдат, свой дом, своя магия. Нельзя смешивать: орк в человеческом доме — баг.

Идея:
Создать фабрику для целой семьи взаимосвязанных объектов.

Псевдокод:

```
интерфейс РасовыйНабор
   метод СоздатьСолдата()
   метод СоздатьЗдание()
   метод СоздатьМагию()

класс ЛюдскойНабор : РасовыйНабор
   метод СоздатьСолдата() -> new ЧеловекСолдат()
   метод СоздатьЗдание() -> new ЧеловекДом()

класс ОркскийНабор : РасовыйНабор
   метод СоздатьСолдата() -> new ОркСолдат()
   метод СоздатьЗдание() -> new ОркДом()
```

C++:

```
// Продукты
class Soldier { public: virtual void Attack() = 0; };
class Building { public: virtual void Build() = 0; };

class HumanSoldier : public Soldier {
public: void Attack() override { cout << "Рыцарь атакует!" << endl; }
};

class OrcSoldier : public Soldier {
public: void Attack() override { cout << "Орк рубит топором!" << endl; }
};

class HumanHouse : public Building {
public: void Build() override { cout << "Каменный дом построен" << endl; }
};

class OrcHut : public Building {
public: void Build() override { cout << "Хижина из шкур" << endl; }
};

// Абстрактная фабрика
class RaceFactory {
public:
   virtual Soldier* CreateSoldier() = 0;
   virtual Building* CreateBuilding() = 0;
};

class HumanFactory : public RaceFactory {
public:
   Soldier* CreateSoldier() override { return new HumanSoldier(); }
   Building* CreateBuilding() override { return new HumanHouse(); }
};

class OrcFactory : public RaceFactory {
public:
   Soldier* CreateSoldier() override { return new OrcSoldier(); }
   Building* CreateBuilding() override { return new OrcHut(); }
};

// Использование
RaceFactory* factory = new HumanFactory();
Soldier* s = factory->CreateSoldier();
s->Attack(); // "Рыцарь атакует!"
```

C#:

```
// Интерфейсы
interface ISoldier { void Attack(); }
interface IBuilding { void Build(); }
interface IRaceFactory
{
    ISoldier CreateSoldier();
    IBuilding CreateBuilding();
}

// Реализация
class HumanFactory : IRaceFactory
{
    public ISoldier CreateSoldier() => new HumanSoldier();
    public IBuilding CreateBuilding() => new HumanHouse();
}

// Использование
IRaceFactory factory = new HumanFactory();
ISoldier soldier = factory.CreateSoldier();
soldier.Attack();
```

2.3. Строитель (Builder)

Проблема:
Создание сложного персонажа требует 10+ параметров. Конструктор с 15 аргументами — ад.

```
// Плохо
Hero hero = new Hero("Артас", 100, 50, 20, 15, 30, true, false, "Меч", "Щит", null);
```

Идея:
Вынести конструирование в отдельный класс, добавляя параметры пошагово.

Псевдокод:

```
строитель = new ГеройСтроитель()
строитель.УстановитьИмя("Артас")
строитель.УстановитьБроню("Ледяная")
строитель.ДобавитьСпособность("Ледяной удар")
герой = строитель.Построить()
```

C++:

```
class Hero {
private:
    string name;
    int hp, mana;
    string weapon, armor;

public:
    // Конструктор закрыт
    Hero(string n, int h, int m, string w, string a)
        : name(n), hp(h), mana(m), weapon(w), armor(a) {}

    class Builder {
    private:
        string name = "Безымянный";
        int hp = 100;
        int mana = 50;
        string weapon = "Кулаки";
        string armor = "Тряпка";

    public:
        Builder& SetName(string n) { name = n; return *this; }
        Builder& SetHP(int h) { hp = h; return *this; }
        Builder& SetWeapon(string w) { weapon = w; return *this; }
        Builder& SetArmor(string a) { armor = a; return *this; }

        Hero Build() {
            return Hero(name, hp, mana, weapon, armor);
        }
    };
};

// Использование
Hero hero = Hero::Builder()
            .SetName("Артас")
            .SetHP(200)
            .SetWeapon("Ледяная Скорбь")
            .SetArmor("Некропласт")
            .Build();
```

C#:

```
class HeroBuilder
{
    private string _name = "Безымянный";
    private int _hp = 100;
```

```
    private string _weapon = "Кулаки";

    public HeroBuilder SetName(string name) { _name = name; return this; }
    public HeroBuilder SetHP(int hp) { _hp = hp; return this; }
    public HeroBuilder SetWeapon(string weapon) { _weapon = weapon; return
this; }

    public Hero Build()
    {
        return new Hero(_name, _hp, _weapon);
    }
}

// Использование
Hero hero = new HeroBuilder()
    .SetName("Артас")
    .SetHP(200)
    .SetWeapon("Ледяная Скорбь")
    .Build();
```

2.4. Одиночка (Singleton)

Проблема:
В игре должен быть ровно один менеджер звуков. Если создать второй — звуки наложатся или упадут.

Идея:
Класс сам контролирует свое существование — только один экземпляр.

Псевдокод:

```
класс ЗвукМенеджер
    приватное статическое поле экземпляр

    приватный конструктор() { }

    метод статический ПолучитьЭкземпляр()
        если экземпляр == null
            экземпляр = new ЗвукМенеджер()
        возврат экземпляр
```

C++ (не потокобезопасно для простоты):

```
class AudioManager {
private:
    static AudioManager* instance;
```

```
    // Приватный конструктор
    AudioManager() { cout << "Аудио система запущена" << endl; }

public:
    static AudioManager* GetInstance() {
        if (instance == nullptr) {
            instance = new AudioManager();
        }
        return instance;
    }

    void PlaySound(string sound) {
        cout << "Играет звук: " << sound << endl;
    }
};

// Инициализация статического поля
AudioManager* AudioManager::instance = nullptr;

// Использование
AudioManager* audio = AudioManager::GetInstance();
audio->PlaySound("Взрыв");
// AudioManager* audio2 = new AudioManager(); // Ошибка! конструктор
приватный
```

C#:

```
class AudioManager
{
    private static AudioManager _instance;
    private static readonly object _lock = new object();

    private AudioManager()
    {
        Console.WriteLine("Аудио система запущена");
    }

    public static AudioManager GetInstance()
    {
        if (_instance == null)
        {
            lock (_lock) // Потокобезопасность
            {
                if (_instance == null)
```

```
                _instance = new AudioManager();
            }
        }
        return _instance;
    }

    public void PlaySound(string sound)
    {
        Console.WriteLine($"Играет звук: {sound}");
    }
}

// Использование
var audio = AudioManager.GetInstance();
audio.PlaySound("Взрыв");
```

2.5. Прототип (Prototype)

Проблема:
Создание копии монстра через new занимает время. А в игре нужно быстро клонировать армию.

Идея:
Объект сам себя клонирует (не через конструктор).

Псевдокод:

```
класс Монстр
    поле здоровье
    поле урон

    метод Клонировать()
        возврат копия этого объекта
```

C++:

```
class Monster {
private:
    int health;
    int damage;
    string name;

public:
    Monster(string n, int h, int d) : name(n), health(h), damage(d) {}

    // Конструктор копирования
```

```
    Monster(const Monster& other)
        : name(other.name), health(other.health), damage(other.damage) {}

    virtual Monster* Clone() {
        return new Monster(*this); // вызывает конструктор копирования
    }

    void Roar() {
        cout << name << " рычит! HP: " << health << endl;
    }
};

// Использование
Monster* orcPrototype = new Monster("Орк", 100, 20);
Monster* orc1 = orcPrototype->Clone();
Monster* orc2 = orcPrototype->Clone();

orc1->Roar(); // Орк рычит! HP: 100
orc2->Roar(); // Орк рычит! HP: 100
```

C#:

```
class Monster : ICloneable
{
    public string Name { get; set; }
    public int Health { get; set; }
    public int Damage { get; set; }

    public Monster(string name, int health, int damage)
    {
        Name = name;
        Health = health;
        Damage = damage;
    }

    public object Clone()
    {
        // Поверхностное копирование (для простых полей)
        return this.MemberwiseClone();
    }

    public void Roar()
    {
        Console.WriteLine($"{Name} рычит! HP: {Health}");
    }
```

```
}

// Использование
Monster orcPrototype = new Monster("Орк", 100, 20);
Monster orc1 = (Monster)orcPrototype.Clone();
Monster orc2 = (Monster)orcPrototype.Clone();
```

ЧАСТЬ 3: СТРУКТУРНЫЕ ПАТТЕРНЫ (Structural)

О том, **как строить связи между классами**.

3.6. Адаптер (Adapter)

Проблема:
У нас старая система боя с методом AttackEnemy(warrior). Новая система работает с IAttacker. Как заставить старых воинов работать в новой системе?

Идея:
Создать класс-прокладку, который переводит вызовы нового интерфейса в старые методы.

Псевдокод:

```
// Старый класс
класс СтарыйВоин
   метод НанестиУдар()

// Адаптер
класс АдаптерВоин implements НовыйИнтерфейс
   поле старыйВоин

   метод Атаковать()
      старыйВоин.НанестиУдар()
```

C++:

```
// Старый класс (не можем менять)
class OldWarrior {
public:
   void SlashWithSword() {
      cout << "Старый воин рубит мечом!" << endl;
   }
};

// Новый интерфейс
```

```
class IAttacker {
public:
    virtual void Attack() = 0;
};

// Адаптер
class WarriorAdapter : public IAttacker {
private:
    OldWarrior* oldWarrior;

public:
    WarriorAdapter(OldWarrior* w) : oldWarrior(w) {}

    void Attack() override {
        oldWarrior->SlashWithSword(); // преобразуем вызов
    }
};

// Новая система
class NewGameEngine {
public:
    void Fight(IAttacker* attacker) {
        attacker->Attack();
    }
};

// Использование
OldWarrior* oldGuy = new OldWarrior();
IAttacker* adapter = new WarriorAdapter(oldGuy);

NewGameEngine engine;
engine.Fight(adapter); // "Старый воин рубит мечом!"
```

C#:

```
// Старый класс
class OldWarrior
{
    public void SlashWithSword()
    {
        Console.WriteLine("Старый воин рубит мечом!");
    }
}

// Новый интерфейс
```

```
interface IAttacker
{
   void Attack();
}

// Адаптер
class WarriorAdapter : IAttacker
{
   private OldWarrior _oldWarrior;

   public WarriorAdapter(OldWarrior oldWarrior)
   {
      _oldWarrior = oldWarrior;
   }

   public void Attack()
   {
      _oldWarrior.SlashWithSword();
   }
}

// Использование
OldWarrior oldGuy = new OldWarrior();
IAttacker adapter = new WarriorAdapter(oldGuy);
adapter.Attack();
```

3.7. Мост (Bridge)

Проблема:
Есть типы персонажей (Воин, Маг) и виды атак (Физическая, Магическая). Если делать через наследование, получим 2×2 класса: ВоинСФизикой, ВоинСМагией, МагСФизикой, МагСМагией. Добавится 3-й тип атаки — классы взрываются.

Идея:
Разделить иерархии “что” и “как”. Связать их через ссылку.

Псевдокод:

```
класс Персонаж
   поле способАтаки   // ссылка на интерфейс атаки
   метод Атаковать()
      способАтаки.Выполнить()

класс Воин : Персонаж
```

```
    // использует переданную атаку

интерфейс СпособАтаки
    метод Выполнить()

класс ФизикаАтака : СпособАтаки
класс МагияАтака : СпособАтаки
```

C++:

```
// Интерфейс реализации (как атаковать)
class AttackStyle {
public:
    virtual void Execute() = 0;
};

class MeleeAttack : public AttackStyle {
public:
    void Execute() override { cout << "Рубящий удар!" << endl; }
};

class MagicAttack : public AttackStyle {
public:
    void Execute() override { cout << "Файерболл!" << endl; }
};

// Абстракция (кто атакует)
class Character {
protected:
    AttackStyle* attackStyle; // мост

public:
    Character(AttackStyle* style) : attackStyle(style) {}
    virtual void PerformAttack() = 0;
};

class Warrior : public Character {
public:
    Warrior(AttackStyle* style) : Character(style) {}

    void PerformAttack() override {
        cout << "Воин: ";
        attackStyle->Execute();
    }
};
```

```
class Mage : public Character {
public:
    Mage(AttackStyle* style) : Character(style) {}

    void PerformAttack() override {
        cout << "Маг: ";
        attackStyle->Execute();
    }
};

// Использование
AttackStyle* melee = new MeleeAttack();
AttackStyle* magic = new MagicAttack();

Character* warriorWithMelee = new Warrior(melee);
Character* mageWithMagic = new Mage(magic);

warriorWithMelee->PerformAttack(); // Воин: Рубящий удар!
mageWithMagic->PerformAttack();    // Маг: Файерболл!

// Легко комбинировать
Character* warriorWithMagic = new Warrior(magic);
warriorWithMagic->PerformAttack(); // Воин: Файерболл!
```

C#:

```
// Интерфейс реализации
interface IAttackStyle
{
    void Execute();
}

class MeleeAttack : IAttackStyle
{
    public void Execute() => Console.WriteLine("Рубящий удар!");
}

class MagicAttack : IAttackStyle
{
    public void Execute() => Console.WriteLine("Файерболл!");
}

// Абстракция
abstract class Character
```

```
{
    protected IAttackStyle _attackStyle;

    protected Character(IAttackStyle style)
    {
        _attackStyle = style;
    }

    public abstract void PerformAttack();
}

class Warrior : Character
{
    public Warrior(IAttackStyle style) : base(style) { }

    public override void PerformAttack()
    {
        Console.Write("Воин: ");
        _attackStyle.Execute();
    }
}

// Использование аналогично C++
```

3.8. Компоновщик (Composite)

Проблема:
Группа врагов должна вести себя как один враг (получить урон по группе, перемещать группу).

Идея:
Древовидная структура “часть-целое”. И листья (один враг), и контейнеры (группа) имеют общий интерфейс.

Псевдокод:

```
интерфейс ИгровойОбъект
    метод НанестиУрон(количество)

класс Враг : ИгровойОбъект
    поле здоровье
    метод НанестиУрон(количество) { здоровье -= количество }

класс ГруппаВрагов : ИгровойОбъект
    поле список дети
```

```
    метод НанестиУрон(количество)
        для каждого ребенка в дети
            ребенок.НанестиУрон(количество)
```

C++:

```
#include <vector>
#include <algorithm>

class GameObject {
public:
    virtual void TakeDamage(int amount) = 0;
    virtual ~GameObject() {}
};

// Лист
class Enemy : public GameObject {
private:
    string name;
    int health;

public:
    Enemy(string n, int h) : name(n), health(h) {}

    void TakeDamage(int amount) override {
        health -= amount;
        cout << name << " получил урон. Осталось HP: " << health << endl;
    }
};

// Композит
class EnemyGroup : public GameObject {
private:
    string groupName;
    vector<GameObject*> members;

public:
    EnemyGroup(string name) : groupName(name) {}

    void Add(GameObject* obj) {
        members.push_back(obj);
    }

    void Remove(GameObject* obj) {
        members.erase(remove(members.begin(), members.end(), obj),
```

```
members.end());
    }

    void TakeDamage(int amount) override {
        cout << "Группа " << groupName << " получает урон!" << endl;
        for (GameObject* obj : members) {
            obj->TakeDamage(amount);
        }
    }
};

// Использование
Enemy* goblin1 = new Enemy("Гоблин-1", 50);
Enemy* goblin2 = new Enemy("Гоблин-2", 50);
Enemy* boss = new Enemy("Босс", 200);

EnemyGroup* patrol = new EnemyGroup("Патруль");
patrol->Add(goblin1);
patrol->Add(goblin2);

EnemyGroup* dungeon = new EnemyGroup("Подземелье");
dungeon->Add(patrol); // группа в группе!
dungeon->Add(boss);

// Один вызов - урон по всей структуре
dungeon->TakeDamage(30);
```

C#:

```
using System.Collections.Generic;

interface IGameObject
{
    void TakeDamage(int amount);
}

class Enemy : IGameObject
{
    private string _name;
    private int _health;

    public Enemy(string name, int health)
    {
        _name = name;
        _health = health;
```

```
    }

    public void TakeDamage(int amount)
    {
        _health -= amount;
        Console.WriteLine($"{_name} получил урон. Осталось HP: {_health}");
    }
}

class EnemyGroup : IGameObject
{
    private string _groupName;
    private List<IGameObject> _members = new List<IGameObject>();

    public EnemyGroup(string name)
    {
        _groupName = name;
    }

    public void Add(IGameObject obj) => _members.Add(obj);
    public void Remove(IGameObject obj) => _members.Remove(obj);

    public void TakeDamage(int amount)
    {
        Console.WriteLine($"Группа {_groupName} получает урон!");
        foreach (var obj in _members)
        {
            obj.TakeDamage(amount);
        }
    }
}
```

3.9. Декоратор (Decorator)

Проблема:
Нужно динамически добавлять способности герою: щит, невидимость, огненный меч. Через наследование получим комбинаторный взрыв классов.

Идея:
Обернуть объект в другой объект, который добавляет новое поведение.

Псевдокод:

```
интерфейс Герой
    метод ПолучитьУрон(урон)
```

```
класс ОбычныйГерой : Герой

класс ДекораторЩита : Герой
   поле декорируемыйГерой
   метод ПолучитьУрон(урон)
      если есть щит, уменьшить урон
      декорируемыйГерой.ПолучитьУрон(новыйУрон)
```

C++:

```
class Hero {
public:
   virtual void TakeDamage(int damage) = 0;
   virtual int GetArmor() = 0;
};

class BasicHero : public Hero {
private:
   string name;
   int health;

public:
   BasicHero(string n, int h) : name(n), health(h) {}

   void TakeDamage(int damage) override {
      health -= damage;
      cout << name << " получил " << damage << " урона. HP: " << health <<
endl;
   }

   int GetArmor() override { return 0; }
};

// Базовый декоратор
class HeroDecorator : public Hero {
protected:
   Hero* wrappedHero;

public:
   HeroDecorator(Hero* hero) : wrappedHero(hero) {}

   void TakeDamage(int damage) override {
      wrappedHero->TakeDamage(damage);
   }
```

```
    int GetArmor() override {
        return wrappedHero->GetArmor();
    }
};

// Конкретный декоратор: Щит
class ShieldDecorator : public HeroDecorator {
private:
    int armorBonus = 20;

public:
    ShieldDecorator(Hero* hero) : HeroDecorator(hero) {}

    void TakeDamage(int damage) override {
        int reducedDamage = max(0, damage - armorBonus);
        cout << "Щит поглотил " << min(damage, armorBonus) << " урона!" <<
endl;
        wrappedHero->TakeDamage(reducedDamage);
    }

    int GetArmor() override {
        return wrappedHero->GetArmor() + armorBonus;
    }
};

// Конкретный декоратор: Невидимость
class InvisibilityDecorator : public HeroDecorator {
public:
    InvisibilityDecorator(Hero* hero) : HeroDecorator(hero) {}

    void TakeDamage(int damage) override {
        // 50% шанс избежать урона
        if (rand() % 2 == 0) {
            cout << "Невидимость! Атака промахнулась!" << endl;
            return;
        }
        wrappedHero->TakeDamage(damage);
    }
};

// Использование
Hero* hero = new BasicHero("Артас", 100);
hero = new ShieldDecorator(hero);       // добавили щит
```

```
hero = new InvisibilityDecorator(hero); // добавили невидимость

hero->TakeDamage(50); // Щит уменьшит, невидимость может избежать
```

C#:

```
using System;

interface IHero
{
    void TakeDamage(int damage);
    int GetArmor();
}

class BasicHero : IHero
{
    private string _name;
    private int _health;

    public BasicHero(string name, int health)
    {
        _name = name;
        _health = health;
    }

    public void TakeDamage(int damage)
    {
        _health -= damage;
        Console.WriteLine($"{_name} получил {damage} урона. HP: {_health}");
    }

    public int GetArmor() => 0;
}

abstract class HeroDecorator : IHero
{
    protected IHero _wrappedHero;

    public HeroDecorator(IHero hero)
    {
        _wrappedHero = hero;
    }

    public virtual void TakeDamage(int damage) =>
_wrappedHero.TakeDamage(damage);
```

```
    public virtual int GetArmor() => _wrappedHero.GetArmor();
}

class ShieldDecorator : HeroDecorator
{
    private int _armorBonus = 20;

    public ShieldDecorator(IHero hero) : base(hero) { }

    public override void TakeDamage(int damage)
    {
        int reducedDamage = Math.Max(0, damage - _armorBonus);
        Console.WriteLine($"Щит поглотил {Math.Min(damage, _armorBonus)}
урона!");
        _wrappedHero.TakeDamage(reducedDamage);
    }

    public override int GetArmor() => _wrappedHero.GetArmor() + _armorBonus;
}
```

10. Фасад (Facade)

Проблема:
Чтобы запустить уровень, нужно: загрузить текстуры, инициализировать физику, создать врагов, запустить звук, настроить камеру. Слишком сложно.

Идея:
Создать простой интерфейс для сложной подсистемы.

Псевдокод:

```
класс ЗапускУровняФасад
    метод ЗапуститьУровень(название)
        текстуры.Загрузить()
        физика.Инициализировать()
        враги.Создать()
        звук.Включить()
        камера.Настроить()
```

C++:

```
// Сложные подсистемы
class TextureManager {
public:
    void LoadTextures() { cout << "Текстуры загружены" << endl; }
};
```

```
class PhysicsEngine {
public:
   void Init() { cout << "Физика инициализирована" << endl; }
};

class EnemySpawner {
public:
   void SpawnEnemies() { cout << "Враги созданы" << endl; }
};

class AudioSystem {
public:
   void Start() { cout << "Звук запущен" << endl; }
};

// Фасад
class GameLevelFacade {
private:
   TextureManager textures;
   PhysicsEngine physics;
   EnemySpawner spawner;
   AudioSystem audio;

public:
   void StartLevel(string levelName) {
      cout << "Запуск уровня: " << levelName << endl;
      textures.LoadTextures();
      physics.Init();
      spawner.SpawnEnemies();
      audio.Start();
      cout << "Уровень готов!" << endl;
   }
};

// Использование
GameLevelFacade facade;
facade.StartLevel("Пещера дракона");
```

C#:

```
class GameLevelFacade
{
   private TextureManager _textures = new TextureManager();
   private PhysicsEngine _physics = new PhysicsEngine();
```

```
    private EnemySpawner _spawner = new EnemySpawner();
    private AudioSystem _audio = new AudioSystem();

    public void StartLevel(string levelName)
    {
        Console.WriteLine($"Запуск уровня: {levelName}");
        _textures.LoadTextures();
        _physics.Init();
        _spawner.SpawnEnemies();
        _audio.Start();
        Console.WriteLine("Уровень готов!");
    }
}

// Использование
var facade = new GameLevelFacade();
facade.StartLevel("Пещера дракона");
```

11. Легковес (Flyweight)

Проблема:
В игре 1000 орков. У каждого орка есть моделька, текстура, анимация (тяжелые данные). Если хранить для каждого — память лопнет.

Идея:
Разделить данные на:

- **Внутренние (разделяемые)**: модель, текстура (одинаковые для всех орков)
- **Внешние (уникальные)**: позиция, HP, направление

Псевдокод:

```
класс ТипОрка   // легковес
    поле модель
    поле текстура

класс Орк
    поле тип     // ссылка на разделяемый тип
    поле позиция
    поле здоровье
```

C++:

```
#include <map>
```

```
// Легковес (разделяемые данные)
class OrcType {
private:
    string model;
    string texture;

public:
    OrcType(string m, string t) : model(m), texture(t) {}

    void Render(int x, int y, int hp) {
        cout << "Орк (" << model << ", " << texture << ") на позиции "
            << x << "," << y << " HP:" << hp << endl;
    }
};

// Фабрика легковесов
class OrcTypeFactory {
private:
    map<string, OrcType*> types;

public:
    OrcType* GetOrcType(string model, string texture) {
        string key = model + "_" + texture;

        if (types.find(key) == types.end()) {
            types[key] = new OrcType(model, texture);
            cout << "Создан новый тип орка: " << key << endl;
        }
        return types[key];
    }
};

// Конкретный орк (внешние данные)
class Orc {
private:
    OrcType* type;  // разделяемый
    int x, y;       // уникальные
    int health;

public:
    Orc(OrcType* t, int xPos, int yPos, int hp)
        : type(t), x(xPos), y(yPos), health(hp) {}

    void Render() {
```

```
        type->Render(x, y, health);
    }
};

// Использование
OrcTypeFactory factory;

// 1000 орков с одним типом
vector<Orc*> army;
OrcType* greenOrc = factory.GetOrcType("Базовый", "Зеленая текстура");

for (int i = 0; i < 1000; i++) {
    army.push_back(new Orc(greenOrc, i * 10, 0, 100));
}

// Памяти использовано: 1 тип + 1000 легковесных орков,
// а не 1000 тяжелых орков с дублирующимися данными
```

C#:

```
using System.Collections.Generic;

class OrcType
{
    private string _model;
    private string _texture;

    public OrcType(string model, string texture)
    {
        _model = model;
        _texture = texture;
    }

    public void Render(int x, int y, int hp)
    {
        Console.WriteLine($"Орк ({_model}, {_texture}) на позиции {x},{y}
HP:{hp}");
    }
}

class OrcTypeFactory
{
    private Dictionary<string, OrcType> _types = new Dictionary<string,
OrcType>();
```

```
    public OrcType GetOrcType(string model, string texture)
    {
        string key = $"{model}_{texture}";

        if (!_types.ContainsKey(key))
        {
            _types[key] = new OrcType(model, texture);
            Console.WriteLine($"Создан новый тип орка: {key}");
        }
        return _types[key];
    }
}

class Orc
{
    private OrcType _type;
    private int _x, _y;
    private int _health;

    public Orc(OrcType type, int x, int y, int health)
    {
        _type = type;
        _x = x;
        _y = y;
        _health = health;
    }

    public void Render() => _type.Render(_x, _y, _health);
}
```

12. Заместитель (Proxy)

Проблема:
Загрузка текстуры босса тяжелая. Не хочется грузить ее, пока игрок не подошел к боссу.

Идея:
Создать объект-заменитель, который контролирует доступ к реальному объекту (ленивая загрузка).

Псевдокод:

```
интерфейс Текстура
    метод Отобразить()
```

```
класс РеальнаяТекстура : Текстура   // тяжелая

класс ЗаместительТекстуры : Текстура   // легковес
   поле реальнаяТекстура = null
   метод Отобразить()
      если реальнаяТекстура == null
         реальнаяТекстура = new РеальнаяТекстура()
      реальнаяТекстура.Отобразить()
```

C++:

```
class ITexture {
public:
   virtual void Display() = 0;
};

// Реальный объект (тяжелый)
class BossTexture : public ITexture {
private:
   string filename;

   void LoadFromDisk() {
      cout << "Загрузка тяжелой текстуры: " << filename << endl;
      // тяжелая операция
   }

public:
   BossTexture(string file) : filename(file) {
      LoadFromDisk(); // грузим сразу? не всегда нужно
   }

   void Display() override {
      cout << "Отображение текстуры босса" << endl;
   }
};

// Заместитель (ленивая загрузка)
class TextureProxy : public ITexture {
private:
   BossTexture* realTexture;
   string filename;

public:
   TextureProxy(string file) : filename(file), realTexture(nullptr) {}
```

```
    void Display() override {
        if (realTexture == nullptr) {
            realTexture = new BossTexture(filename); // грузим только когда нужно
        }
        realTexture->Display();
    }
};

// Использование
TextureProxy* bossTex = new TextureProxy("dragon.png");

// Текстура еще не загружена
// ... много кода ...

// Только когда действительно нужно отобразить
bossTex->Display(); // тут загрузится
```

C#:

```
interface ITexture
{
    void Display();
}

class BossTexture : ITexture
{
    private string _filename;

    public BossTexture(string filename)
    {
        _filename = filename;
        LoadFromDisk();
    }

    private void LoadFromDisk()
    {
        Console.WriteLine($"Загрузка тяжелой текстуры: {_filename}");
    }

    public void Display()
    {
        Console.WriteLine("Отображение текстуры босса");
    }
}
```

```
class TextureProxy : ITexture
{
    private BossTexture _realTexture;
    private string _filename;

    public TextureProxy(string filename)
    {
        _filename = filename;
    }

    public void Display()
    {
        if (_realTexture == null)
        {
            _realTexture = new BossTexture(_filename);
        }
        _realTexture.Display();
    }
}
```

ЧАСТЬ 4: ПОВЕДЕНЧЕСКИЕ ПАТТЕРНЫ (Behavioral)

О том, **как объекты взаимодействуют**.

4.13. Цепочка Обязанностей (Chain of Responsibility)

Проблема:
Обработка урона: сначала щит, потом броня, потом здоровье. Кто и в каком порядке должен обрабатывать?

Идея:
Выстроить обработчики в цепочку. Каждый решает, обработать запрос или передать дальше.

Псевдокод:

```
класс ОбработчикУрона
    поле следующий
    метод Обработать(урон)
        если могу обработать:
            обрабатываю
        иначе если следующий есть:
            следующий.Обработать(урон)
```

C++:

```
class DamageHandler {
protected:
   DamageHandler* next;

public:
   DamageHandler() : next(nullptr) {}

   void SetNext(DamageHandler* n) { next = n; }

   virtual void HandleDamage(int& damage) {
      if (next) next->HandleDamage(damage);
   }
};

class ShieldHandler : public DamageHandler {
public:
   void HandleDamage(int& damage) override {
      if (damage > 0) {
         int blocked = min(damage, 20);
         damage -= blocked;
         cout << "Щит заблокировал " << blocked << " урона. Осталось: " <<
damage << endl;
      }
      DamageHandler::HandleDamage(damage);
   }
};

class ArmorHandler : public DamageHandler {
public:
   void HandleDamage(int& damage) override {
      if (damage > 0) {
         int reduced = damage * 0.3; // броня уменьшает на 30%
         damage -= reduced;
         cout << "Броня поглотила " << reduced << " урона. Осталось: " <<
damage << endl;
      }
      DamageHandler::HandleDamage(damage);
   }
};

class HealthHandler : public DamageHandler {
private:
   int health = 100;
```

```
public:
    void HandleDamage(int& damage) override {
        if (damage > 0) {
            health -= damage;
            cout << "Здоровье уменьшено на " << damage << ". Текущее HP: " <<
health << endl;
        }
    }
};

// Использование
ShieldHandler* shield = new ShieldHandler();
ArmorHandler* armor = new ArmorHandler();
HealthHandler* health = new HealthHandler();

shield->SetNext(armor);
armor->SetNext(health);

int damage = 50;
shield->HandleDamage(damage); // пройдет по цепочке
```

C#:

```
abstract class DamageHandler
{
    protected DamageHandler _next;

    public void SetNext(DamageHandler next) => _next = next;

    public virtual void HandleDamage(ref int damage)
    {
        _next?.HandleDamage(ref damage);
    }
}

class ShieldHandler : DamageHandler
{
    public override void HandleDamage(ref int damage)
    {
        if (damage > 0)
        {
            int blocked = Math.Min(damage, 20);
            damage -= blocked;
            Console.WriteLine($"Щит заблокировал {blocked} урона. Осталось:
```

```
{damage}");
        }
        base.HandleDamage(ref damage);
    }
}
```

4.14. Команда (Command)

Проблема:
Нужно сделать Undo/Redo для действий игрока (переместил, ударил, использовал зелье).

Идея:
Превратить запрос в объект. Можно хранить историю, отменять, повторять.

Псевдокод:

```
интерфейс Команда
    метод Выполнить()
    метод Отменить()

класс ПереместитьКоманда : Команда
    поле объект, старыеКоординаты, новыеКоординаты
```

C++:

```
#include <stack>

class Unit {
private:
    int x, y;
    string name;

public:
    Unit(string n, int startX, int startY) : name(n), x(startX), y(startY) {}

    void Move(int newX, int newY) {
        cout << name << " перемещен с (" << x << "," << y << ") на ("
            << newX << "," << newY << ")" << endl;
        x = newX;
        y = newY;
    }

    void SetPosition(int newX, int newY) { x = newX; y = newY; }
    pair<int,int> GetPosition() { return {x, y}; }
};
```

```
class Command {
public:
    virtual void Execute() = 0;
    virtual void Undo() = 0;
};

class MoveCommand : public Command {
private:
    Unit* unit;
    int prevX, prevY;
    int newX, newY;

public:
    MoveCommand(Unit* u, int nx, int ny)
        : unit(u), newX(nx), newY(ny) {}

    void Execute() override {
        auto pos = unit->GetPosition();
        prevX = pos.first;
        prevY = pos.second;
        unit->Move(newX, newY);
    }

    void Undo() override {
        unit->Move(prevX, prevY);
    }
};

// Invoker
class CommandHistory {
private:
    stack<Command*> history;

public:
    void ExecuteCommand(Command* cmd) {
        cmd->Execute();
        history.push(cmd);
    }

    void Undo() {
        if (!history.empty()) {
            Command* cmd = history.top();
            cmd->Undo();
```

```
            history.pop();
            delete cmd;
        }
    }
};

// Использование
Unit* hero = new Unit("Герой", 0, 0);
CommandHistory history;

history.ExecuteCommand(new MoveCommand(hero, 10, 0));
history.ExecuteCommand(new MoveCommand(hero, 10, 10));

history.Undo(); // вернется на 10,0
history.Undo(); // вернется на 0,0
```

C#:

```
interface ICommand
{
    void Execute();
    void Undo();
}

class MoveCommand : ICommand
{
    private Unit _unit;
    private int _prevX, _prevY;
    private int _newX, _newY;

    public MoveCommand(Unit unit, int newX, int newY)
    {
        _unit = unit;
        _newX = newX;
        _newY = newY;
    }

    public void Execute()
    {
        var pos = _unit.GetPosition();
        _prevX = pos.Item1;
        _prevY = pos.Item2;
        _unit.Move(_newX, _newY);
    }
```

```
    public void Undo() => _unit.Move(_prevX, _prevY);
}
```

4.15. Итератор (Iterator)

Проблема:
Нужно обойти инвентарь игрока. Но инвентарь может быть массивом, списком или словарем. Клиент не должен знать структуру.

Идея:
Дать единый способ перебора коллекций, не раскрывая внутреннюю структуру.

Псевдокод:

```
класс Инвентарь
    метод ПолучитьИтератор()

класс Итератор
    метод ЕстьСледующий()
    метод Следующий()
```

C++:

```
class Item {
public:
    string name;
    Item(string n) : name(n) {}
};

// Кастомный итератор
class InventoryIterator {
private:
    vector<Item*>* items;
    int index = 0;

public:
    InventoryIterator(vector<Item*>* i) : items(i) {}

    bool HasNext() {
        return index < items->size();
    }

    Item* Next() {
        return (*items)[index++];
    }
```

```
};

class Inventory {
private:
    vector<Item*> items;

public:
    void AddItem(Item* item) {
        items.push_back(item);
    }

    InventoryIterator* CreateIterator() {
        return new InventoryIterator(&items);
    }
};

// Использование
Inventory* inv = new Inventory();
inv->AddItem(new Item("Меч"));
inv->AddItem(new Item("Щит"));
inv->AddItem(new Item("Зелье"));

InventoryIterator* it = inv->CreateIterator();
while (it->HasNext()) {
    Item* item = it->Next();
    cout << "Предмет: " << item->name << endl;
}
```

C# (встроенный итератор):

```
using System.Collections;

class Inventory : IEnumerable<Item>
{
    private List<Item> _items = new List<Item>();

    public void AddItem(Item item) => _items.Add(item);

    public IEnumerator<Item> GetEnumerator()
    {
        foreach (var item in _items)
            yield return item;
    }

    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
```

```
}

// Использование
Inventory inv = new Inventory();
inv.AddItem(new Item("Меч"));
inv.AddItem(new Item("Щит"));

// foreach использует итератор автоматически
foreach (Item item in inv)
{
    Console.WriteLine($"Предмет: {item.Name}");
}
```

4.16. Посредник (Mediator)

Проблема:
В UI окна, кнопки, списки связаны напрямую. Кнопка “Сохранить” знает про поле имени и поле класса. Слишком много связей.

Идея:
Создать объект-посредник, который координирует взаимодействие.

Псевдокод:

```
класс ОкноПосредник
    поле полеИмени
    поле кнопкаСохранить

    метод КнопкаНажата()
        если полеИмени.валидно()
            сохранить
        иначе
            показатьОшибку()
```

C++:

```
class UIMediator;

class UIComponent {
protected:
    UIMediator* mediator;

public:
    UIComponent(UIMediator* m) : mediator(m) {}
    virtual void Changed() = 0;
};
```

```
class UIMediator {
private:
    class NameField* nameField;
    class SaveButton* saveButton;

public:
    void RegisterNameField(NameField* f) { nameField = f; }
    void RegisterSaveButton(SaveButton* b) { saveButton = b; }

    void Notify(UIComponent* sender, string event);
};

class NameField : public UIComponent {
private:
    string text;

public:
    NameField(UIMediator* m) : UIComponent(m) {}

    void SetText(string t) {
        text = t;
        Changed(); // уведомляем посредника
    }

    string GetText() { return text; }

    void Changed() override {
        mediator->Notify(this, "textChanged");
    }
};

class SaveButton : public UIComponent {
private:
    bool enabled = false;

public:
    SaveButton(UIMediator* m) : UIComponent(m) {}

    void SetEnabled(bool e) {
        enabled = e;
        cout << "Кнопка " << (e ? "доступна" : "недоступна") << endl;
    }
```

```
    void Click() {
        if (enabled) {
            cout << "Сохранение..." << endl;
            Changed();
        }
    }

    void Changed() override {
        mediator->Notify(this, "clicked");
    }
};

void UIMediator::Notify(UIComponent* sender, string event) {
    if (sender == nameField && event == "textChanged") {
        // Валидация поля имени
        bool isValid = nameField->GetText().length() >= 3;
        saveButton->SetEnabled(isValid);
    }
    else if (sender == saveButton && event == "clicked") {
        cout << "Сохранено: " << nameField->GetText() << endl;
    }
}

// Использование
UIMediator* mediator = new UIMediator();
NameField* nameField = new NameField(mediator);
SaveButton* saveButton = new SaveButton(mediator);

mediator->RegisterNameField(nameField);
mediator->RegisterSaveButton(saveButton);

nameField->SetText("A"); // кнопка недоступна
nameField->SetText("Артас"); // кнопка доступна
saveButton->Click(); // сохранение
```

C#:

```
class UIMediator
{
    private NameField _nameField;
    private SaveButton _saveButton;

    public void RegisterNameField(NameField field) => _nameField = field;
    public void RegisterSaveButton(SaveButton button) => _saveButton = button;
```

```
    public void Notify(UIComponent sender, string @event)
    {
        if (sender == _nameField && @event == "TextChanged")
        {
            bool isValid = _nameField.Text.Length >= 3;
            _saveButton.SetEnabled(isValid);
        }
    }
}
```

4.17. Снимок (Memento)

Проблема:
Нужно сохранять состояние игры (Save/Load), но не раскрывать внутреннюю структуру.

Идея:
Объект сам создает снимок своего состояния и восстанавливается из него.

Псевдокод:

```
класс Игрок
    метод Сохранить() -> Снимок
    метод Восстановить(снимок)

класс Снимок
    поля: здоровье, позиция, инвентарь (приватные)
```

C++:

```
class PlayerMemento {
private:
    int health;
    int level;
    string position;

    // Доступ только для Player
    friend class Player;

    PlayerMemento(int h, int l, string p) : health(h), level(l), position(p) {}

public:
    // Можно только получить данные, но не изменить
    int GetHealth() const { return health; }
};
```

```
class Player {
private:
    int health;
    int level;
    string position;

public:
    Player() : health(100), level(1), position("Старт") {}

    void TakeDamage(int d) { health -= d; }
    void LevelUp() { level++; health = 100; }
    void MoveTo(string p) { position = p; }

    void ShowStatus() {
        cout << "HP:" << health << " Уровень:" << level
            << " Позиция:" << position << endl;
    }

    // Создать снимок
    PlayerMemento* Save() {
        return new PlayerMemento(health, level, position);
    }

    // Восстановить из снимка
    void Restore(PlayerMemento* m) {
        health = m->health;
        level = m->level;
        position = m->position;
    }
};

// Caretaker (хранитель)
class SaveSystem {
private:
    PlayerMemento* save;

public:
    void SaveGame(Player* player) {
        save = player->Save();
        cout << "Игра сохранена" << endl;
    }

    void LoadGame(Player* player) {
        if (save) {
```

```
            player->Restore(save);
            cout << "Игра загружена" << endl;
        }
    }
};

// Использование
Player* player = new Player();
SaveSystem* saves = new SaveSystem();

player->MoveTo("Пещера");
player->TakeDamage(30);
player->ShowStatus(); // HP:70 Уровень:1 Позиция:Пещера

saves->SaveGame(player);

player->LevelUp();
player->MoveTo("Замок");
player->TakeDamage(50);
player->ShowStatus(); // HP:50 Уровень:2 Позиция:Замок

saves->LoadGame(player);
player->ShowStatus(); // HP:70 Уровень:1 Позиция:Пещера
```

C#:

```
class PlayerMemento
{
    public int Health { get; private set; }
    public int Level { get; private set; }
    public string Position { get; private set; }

    public PlayerMemento(int health, int level, string position)
    {
        Health = health;
        Level = level;
        Position = position;
    }
}

class Player
{
    private int _health;
    private int _level;
    private string _position;
```

```
    public Player()
    {
        _health = 100;
        _level = 1;
        _position = "Старт";
    }

    public PlayerMemento Save() => new PlayerMemento(_health, _level,
_position);

    public void Restore(PlayerMemento m)
    {
        _health = m.Health;
        _level = m.Level;
        _position = m.Position;
    }
}
```

4.18. Наблюдатель (Observer)

Проблема:
Когда здоровье игрока меняется, нужно обновить: полоску HP, интерфейс, звук биения сердца, логику врагов.

Идея:
Объект (Subject) рассылает уведомления подписчикам (Observers).

Псевдокод:

```
класс Игрок
    список наблюдатели

    метод ДобавитьНаблюдателя(н)
    метод УдалитьНаблюдателя(н)
    метод УведомитьВсех()

интерфейс Наблюдатель
    метод Обновить(здоровье)
```

C++:

```
#include <vector>

class Observer {
public:
```

```
    virtual void Update(int health) = 0;
};

class Player {
private:
    int health;
    vector<Observer*> observers;

public:
    Player() : health(100) {}

    void Attach(Observer* o) {
        observers.push_back(o);
    }

    void Detach(Observer* o) {
        // удаление из вектора
    }

    void Notify() {
        for (Observer* o : observers) {
            o->Update(health);
        }
    }

    void TakeDamage(int damage) {
        health -= damage;
        cout << "Игрок получил урон! HP: " << health << endl;
        Notify(); // все подписчики узнают
    }
};

class HealthBar : public Observer {
public:
    void Update(int health) override {
        cout << "Полоска здоровья обновлена: "
             << string(health/10, '|') << endl;
    }
};

class HeartbeatSound : public Observer {
public:
    void Update(int health) override {
        if (health < 30) {
```

```
            cout << "*БИЕНИЕ СЕРДЦА* (игрок при смерти!)" << endl;
        }
    }
};

class EnemyAI : public Observer {
public:
    void Update(int health) override {
        if (health < 20) {
            cout << "Враг: ОН ПОЧТИ МЕРТВ! АТАКУЙ!" << endl;
        }
    }
};

// Использование
Player* player = new Player();
HealthBar* hb = new HealthBar();
HeartbeatSound* sound = new HeartbeatSound();
EnemyAI* ai = new EnemyAI();

player->Attach(hb);
player->Attach(sound);
player->Attach(ai);

player->TakeDamage(30); // все подписчики получат уведомление
player->TakeDamage(60); // игрок при смерти
```

C#:

```
using System.Collections.Generic;

interface IObserver
{
    void Update(int health);
}

class Player
{
    private int _health = 100;
    private List<IObserver> _observers = new List<IObserver>();

    public void Attach(IObserver observer) => _observers.Add(observer);

    private void Notify()
    {
```

```
        foreach (var o in _observers)
            o.Update(_health);
    }

    public void TakeDamage(int damage)
    {
        _health -= damage;
        Console.WriteLine($"Игрок получил урон! HP: {_health}");
        Notify();
    }
}

class HealthBar : IObserver
{
    public void Update(int health)
    {
        Console.WriteLine($"Полоска здоровья: {new string('|', health / 10)}");
    }
}
```

4.19. Состояние (State)

Проблема:
Поведение врага зависит от состояния (Спокойный, Агрессивный, Паника). If-ы расползаются по коду.

Идея:
Вынести поведение в отдельные классы состояний. Объект делегирует текущему состоянию.

Псевдокод:

```
класс Враг
    поле состояние

    метод Обновить()
        состояние.Выполнить()
    метод СменитьСостояние(новое)

класс Состояние
    метод Выполнить()
    метод Войти() / Выйти()
```

C++:

```
class Enemy;
class State {
protected:
    Enemy* enemy;

public:
    virtual ~State() {}
    virtual void Enter() = 0;
    virtual void Execute() = 0;
    virtual void Exit() = 0;
    void SetEnemy(Enemy* e) { enemy = e; }
};

class Enemy {
private:
    State* currentState;

public:
    string name;

    Enemy(string n) : name(n), currentState(nullptr) {}

    void ChangeState(State* newState) {
        if (currentState) {
            currentState->Exit();
            delete currentState;
        }
        currentState = newState;
        currentState->SetEnemy(this);
        currentState->Enter();
    }

    void Update() {
        if (currentState) currentState->Execute();
    }
};

// Состояния
class IdleState : public State {
public:
    void Enter() override {
        cout << enemy->name << ": Осматривается" << endl;
    }
```

```
    void Execute() override {
        cout << enemy->name << ": Стоит на месте" << endl;
        // Если видит игрока -> перейти в AggressiveState
    }

    void Exit() override {
        cout << enemy->name << ": Заметил игрока!" << endl;
    }
};

class AggressiveState : public State {
public:
    void Enter() override {
        cout << enemy->name << ": АТАКУЮ!" << endl;
    }

    void Execute() override {
        cout << enemy->name << ": Бежит к игроку" << endl;
        // Если HP низкое -> перейти в PanicState
    }

    void Exit() override {
        cout << enemy->name << ": Отступает..." << endl;
    }
};

class PanicState : public State {
public:
    void Enter() override {
        cout << enemy->name << ": В ПАНИКЕ!" << endl;
    }

    void Execute() override {
        cout << enemy->name << ": Убегает!" << endl;
    }

    void Exit() override {}
};

// Использование
Enemy* goblin = new Enemy("Гоблин");
goblin->ChangeState(new IdleState());

for (int i = 0; i < 3; i++) {
```

```
    goblin->Update();
}

goblin->ChangeState(new AggressiveState());
goblin->Update();
goblin->ChangeState(new PanicState());
goblin->Update();
```

C#:

```
abstract class State
{
    protected Enemy _enemy;
    public void SetEnemy(Enemy e) => _enemy = e;

    public abstract void Enter();
    public abstract void Execute();
    public abstract void Exit();
}

class Enemy
{
    private State _currentState;
    public string Name { get; set; }

    public Enemy(string name) => Name = name;

    public void ChangeState(State newState)
    {
        _currentState?.Exit();
        _currentState = newState;
        _currentState.SetEnemy(this);
        _currentState.Enter();
    }

    public void Update() => _currentState?.Execute();
}
```

4.20. Шаблонный Метод (Template Method)

Проблема:
Все уровни загружаются одинаково: Загрузить текстуры, Создать врагов, Запустить музыку. Но детали отличаются.

Идея:
Определить скелет алгоритма в базовом классе, а шаги делегировать наследникам.

Псевдокод:

```
класс Уровень
   метод ЗагрузитьУровень()   // шаблонный
      ЗагрузитьТекстуры()    // абстрактный
      СоздатьВрагов()        // абстрактный
      ЗапуститьМузыку()      // может быть общим

   абстрактные методы выше
```

C++:

```
class Level {
protected:
   virtual void LoadTextures() = 0;
   virtual void SpawnEnemies() = 0;

   void PlayBackgroundMusic() {
      cout << "Играет фоновая музыка" << endl;
   }

public:
   // Шаблонный метод
   void LoadLevel() {
      cout << "НАЧАЛО ЗАГРУЗКИ УРОВНЯ" << endl;
      LoadTextures();
      SpawnEnemies();
      PlayBackgroundMusic();
      cout << "УРОВЕНЬ ГОТОВ!" << endl;
   }

   virtual ~Level() {}
};

class ForestLevel : public Level {
protected:
   void LoadTextures() override {
      cout << "Загрузка текстур леса: деревья, трава, камни" << endl;
   }

   void SpawnEnemies() override {
      cout << "Спавн гоблинов и эльфов" << endl;
```

```
    }
};

class DungeonLevel : public Level {
protected:
    void LoadTextures() override {
        cout << "Загрузка текстур подземелья: стены, факелы, сундуки" << endl;
    }

    void SpawnEnemies() override {
        cout << "Спавн скелетов и зомби" << endl;
    }
};

// Использование
Level* level = new ForestLevel();
level->LoadLevel(); // единый алгоритм, разная реализация шагов

delete level;
level = new DungeonLevel();
level->LoadLevel();
```

C#:

```
abstract class Level
{
    protected abstract void LoadTextures();
    protected abstract void SpawnEnemies();

    protected virtual void PlayBackgroundMusic()
    {
        Console.WriteLine("Играет фоновая музыка");
    }

    // Шаблонный метод
    public void LoadLevel()
    {
        Console.WriteLine("НАЧАЛО ЗАГРУЗКИ УРОВНЯ");
        LoadTextures();
        SpawnEnemies();
        PlayBackgroundMusic();
        Console.WriteLine("УРОВЕНЬ ГОТОВ!");
    }
}
```

```
class ForestLevel : Level
{
    protected override void LoadTextures()
    {
        Console.WriteLine("Загрузка текстур леса: деревья, трава, камни");
    }

    protected override void SpawnEnemies()
    {
        Console.WriteLine("Спавн гоблинов и эльфов");
    }
}
```

.

21. Посетитель (Visitor)

Проблема:
Нужно добавить новую операцию (например, рендеринг в 3D) для всех типов объектов (Игрок, Враг, Сундук), но нельзя менять их код.

Идея:
Вынести операцию в отдельный класс-посетитель, который “посещает” каждый тип объекта.

Псевдокод:

```
интерфейс Посетитель
    метод ПосетитьИгрока(Игрок)
    метод ПосетитьВрага(Враг)

интерфейс ИгровойОбъект
    метод Принять(Посетитель)

класс Игрок : ИгровойОбъект
    метод Принять(посетитель)
        посетитель.ПосетитьИгрока(этот)
```

C++:

```
class Visitor;

class GameObject {
public:
    virtual void Accept(Visitor* visitor) = 0;
};

class Player;
```

```
class Enemy;
class Chest;

class Visitor {
public:
    virtual void VisitPlayer(Player* player) = 0;
    virtual void VisitEnemy(Enemy* enemy) = 0;
    virtual void VisitChest(Chest* chest) = 0;
};

// Элементы
class Player : public GameObject {
public:
    string name = "Герой";
    int health = 100;

    void Accept(Visitor* visitor) override {
        visitor->VisitPlayer(this);
    }
};

class Enemy : public GameObject {
public:
    string type = "Орк";
    int damage = 20;

    void Accept(Visitor* visitor) override {
        visitor->VisitEnemy(this);
    }
};

class Chest : public GameObject {
public:
    string content = "Золото";
    bool isLocked = false;

    void Accept(Visitor* visitor) override {
        visitor->VisitChest(this);
    }
};

// Конкретный посетитель: Рендеринг
class RenderVisitor : public Visitor {
public:
```

```
    void VisitPlayer(Player* player) override {
        cout << "Рендеринг игрока: " << player->name
            << " с HP:" << player->health << endl;
    }

    void VisitEnemy(Enemy* enemy) override {
        cout << "Рендеринг врага: " << enemy->type
            << " с уроном:" << enemy->damage << endl;
    }

    void VisitChest(Chest* chest) override {
        cout << "Рендеринг сундука: внутри " << chest->content
            << (chest->isLocked ? " (заперт)" : "") << endl;
    }
};

// Другой посетитель: Сохранение в файл
class SaveVisitor : public Visitor {
public:
    void VisitPlayer(Player* player) override {
        cout << "Сохранение игрока в файл" << endl;
    }

    void VisitEnemy(Enemy* enemy) override {
        cout << "Сохранение врага в файл" << endl;
    }

    void VisitChest(Chest* chest) override {
        cout << "Сохранение сундука в файл" << endl;
    }
};

// Использование
vector<GameObject*> objects;
objects.push_back(new Player());
objects.push_back(new Enemy());
objects.push_back(new Chest());

RenderVisitor renderer;
SaveVisitor saver;

// Рендерим все объекты
for (auto obj : objects) {
    obj->Accept(&renderer);
```

```
}

// Сохраняем все объекты
for (auto obj : objects) {
    obj->Accept(&saver);
}
```

C#:

```
interface IVisitor
{
    void VisitPlayer(Player player);
    void VisitEnemy(Enemy enemy);
    void VisitChest(Chest chest);
}

interface IGameObject
{
    void Accept(IVisitor visitor);
}

class Player : IGameObject
{
    public string Name { get; set; } = "Герой";
    public int Health { get; set; } = 100;

    public void Accept(IVisitor visitor) => visitor.VisitPlayer(this);
}

class Enemy : IGameObject
{
    public string Type { get; set; } = "Орк";
    public int Damage { get; set; } = 20;

    public void Accept(IVisitor visitor) => visitor.VisitEnemy(this);
}

class RenderVisitor : IVisitor
{
    public void VisitPlayer(Player player)
    {
        Console.WriteLine($"Рендеринг игрока: {player.Name} с
HP:{player.Health}");
    }
```

```
    public void VisitEnemy(Enemy enemy)
    {
        Console.WriteLine($"Рендеринг врага: {enemy.Type} с
уроном:{enemy.Damage}");
    }

    public void VisitChest(Chest chest)
    {
        Console.WriteLine($"Рендеринг сундука: внутри {chest.Content}");
    }
}
```

.

ИТОГОВАЯ ТАБЛИЦА “ПРОБЛЕМА → ПАТТЕРН” ДЛЯ ИГР

Проблема в игре	Паттерн
Создаем врагов, но не хотим if-ы по типам	**Factory Method**
Целая раса объектов (люди/орки)	**Abstract Factory**
Сложный персонаж с кучей параметров	**Builder**
Должен быть ровно один AudioManager	**Singleton**
Клонирование армии монстров	**Prototype**
Старый код не работает с новым интерфейсом	**Adapter**
Комбинации типов персонажей и атак	**Bridge**
Группа врагов ведет себя как один	**Composite**
Динамически добавить способности герою	**Decorator**
Сложный запуск уровня	**Facade**
1000 орков съедают память	**Flyweight**
Ленивая загрузка тяжелых текстур	**Proxy**
Цепочка обработки урона	**Chain of Resp.**
Undo/Redo для действий игрока	**Command**
Обход инвентаря без знания структуры	**Iterator**
UI элементы не должны знать друг о друге	**Mediator**
Save/Load без нарушения инкапсуляции	**Memento**
Обновление UI при изменении HP	**Observer**
Поведение врага зависит от состояния	**State**
Общий алгоритм загрузки уровней	**Template Method**
Новая операция для всех типов объектов	**Visitor**

КЛЮЧЕВЫЕ ПРИНЦИПЫ ОБУЧЕНИЯ (Повторение)

- **Сначала проблема**, потом код
- **Аналогии из жизни/игр** для запоминания
- **Псевдокод** → C++ → C# (идея важнее синтаксиса)
- **Мини-задача сразу** после объяснения
- **Антипаттерн** (лестница if) перед решением
- **Игровая тематика** для интереса

Эта схема позволяет студентам не заучивать код, а **понимать, когда и зачем** применять каждый паттерн.